

Navigateur IA : plateforme intelligente modulaire embarquée dans un casque de réalité assistée

Travail de fin d'études — Licence en Sciences Informatiques (IA)

Mulongoy Lubabila Adriel

Dir. : Prof. Antoine Bagula Codir. : Ass. Landry Mbale

Juillet 2026

- **2,2 milliards** de personnes avec déficience visuelle (OMS); la canne blanche ignore les obstacles en hauteur et n'apporte aucune analyse sémantique.
- L'IA embarquée légère (TinyML) rend la perception possible sur du matériel grand public.
- **Limite des solutions existantes** : monolithiques, mono-tâche, souvent dépendantes du cloud.

Question scientifique

Comment concevoir une plateforme d'assistance portée, sur matériel embarqué contraint, à latence compatible avec la sécurité, **modulaire, reconfigurable** et **100 % hors ligne** ?

Quatre tensions à résoudre

1. **Ressources vs temps réel** — alerte utile sous ~ 200 ms sur nano-ordinateur.
2. **Monolithe vs modularité** — une panne ne doit pas tout arrêter.
3. **Usage unique vs multi-contexte** — malvoyance, maintenance, formation : un seul casque.
4. **Réseau vs autonomie** — tunnel, sous-sol : la chaîne doit fonctionner hors ligne.

Positionnement face à l'état de l'art

	YOLO embarqué	LLM distant	Nova (proposé)
Latence	E2E 583 ms	2–5 s	bus 2,64 ms (méd.)
Hors ligne	oui	non	oui, 100 %
Modularité	non	non	plugins dynamiques
Multi-usage	non	non	profils configurables

Lacune identifiée : aucun travail ne réunit modularité + hors ligne + reconfigurabilité sur dispositif porté à faible coût.

Architecture Nova en quatre couches

- **Matériel** : Pi 4B (orchestrateur), Arduino Uno (capteurs temps réel, PWM haptique), ESP32-CAM (MJPEG Wi-Fi).
- **Communication** : bus MQTT (Mosquitto, port 1883), publish/subscribe.
- **Applicative** : modules-greffons chargés dynamiquement (`config.json` + `main.py` + interface).
- **Présentation** : interface web Flask en réseau local.



[Insérer Figure 3.2 :
architecture matérielle
du mémoire]

1. **Construire** un artefact réel : prototype physique intégré (Pi 4B + Arduino + ESP32-CAM + capteurs/actionneurs).
2. **Mesurer** sur la cible réelle, pas une machine de dev : Pi 4B Rev 1.5, Linux 6.12, Python 3.11.2.
3. **Protocole reproductible** : 10 campagnes × 60 s, échantillonnage psutil 1 Hz, 100 messages MQTT horodatés/campagne.

Jeux de données : **800 séquences** de gestes, **10 testeurs**, 3 sessions, 3 distances (30/50/80 cm), 3 niveaux de luminosité.

Résultats (1/2) — l'architecture tient ses promesses

Objectif	Mesure réelle	Statut
Latence bus MQTT	2,64 ms médiane, 0 % perte	Atteint
Hors ligne	100 % local	Atteint
Mémoire	29,4 % ($\approx$ 1,18 Go)	Atteint
Thermique	56,4 °C max (marge 24 °C)	Atteint
Autonomie	7 h 30 / cible 8 h	Proche (−6 %)

Reconnaissance vocale (Vosk fr) : **WER 14,3 %** — OCR (Tesseract 5) : **87 %** en conditions favorables.

Résultats (2/2) — résultats négatifs assumés

- **Débit vision : 1,72 FPS** (pipeline complet 640×480) — CPU saturé à **94 %** (pic 97,7 %).
- Compatible avec la commande gestuelle (maintien 0,8 s) mais insuffisant pour un suivi fluide.
- **Latence E2E non instrumentée** : seule la composante transport (2,64 ms) est mesurée; estimation théorique très en deçà de 200 ms (Arduino lu toutes les 10 ms, alerte en $O(1)$) — **mesure prioritaire des travaux futurs**.

Enseignement clé

Le facteur limitant est le **calcul**, pas la chaleur : 94 % CPU mais seulement 56,4 °C. Optimiser la charge de calcul, pas la dissipation.

Scientifiques

- Modèle d'architecture modulaire orchestrée (plugins découverts au démarrage).
- Démonstration d'une chaîne d'assistance multimodale **100 % hors ligne** sur nano-ordinateur.
- Cadre d'évaluation reproductible + **hiérarchie des contraintes** transférable (calcul $\gg$ bus, RAM, thermique).

Techniques

- Prototype intégré documenté (Pi 4B, Arduino, ESP32-CAM, Python/Flask/MQTT).
- Commande multimodale contextuelle (geste + joystick) et profils utilisateurs persistants.

- **Validation utilisateur limitée** : 10 testeurs voyants; pas encore d'étude avec utilisateurs malvoyants (nécessite approbation éthique).
- **CPU** : deux modules d'inférence simultanés maximum sur Pi 4B.
- **Autonomie** : 7 h 30 < cible 8 h; leviers identifiés (cpufreq, veille des modules) non implémentés.
- **Robustesse par injection de panne** : établie par conception, vérification expérimentale à mener.
- Vosk sensible au bruit de fond (pas de module de suppression de bruit).

Court terme

- Instrumenter la latence E2E (priorité n°1).
- Profilage par étape du pipeline vision.
- cpufreq + veille $\Rightarrow$ franchir les 8 h.

Moyen terme

- Migration Raspberry Pi 5 ($\approx 2\times$ plus rapide).
- Déchargement des modèles lourds (mode hors ligne par défaut).
- Étude avec utilisateurs malvoyants (éthique).

- La question posée reçoit une réponse **mesurée et nuancée** : l'architecture modulaire, hors ligne et reconfigurable est **réalisable** sur matériel contraint — et son prix exact est connu (débit vision limité par le calcul).
- Trois tensions sur quatre **résolues**; la quatrième (temps réel vision) partiellement.
- Livrables : prototype fonctionnel, banc d'évaluation reproductible, grille de décision réutilisable.

Merci de votre attention.

Questions ?